

Universitatea Tehnică „Gheorghe Asachi” din Iași

DroidGuard

Sistem distribuit pentru analiza statică a aplicațiilor Android

Student: Solomon Anastasia

Grupa: 1311B

Cuprins

1	Introducere	2
1.1	Scopul documentului	2
1.2	Scurtă descriere a proiectului	2
2	Descriere generala	3
2.1	Contextul sistemului	3
2.2	Modelarea și persistența datelor	5
2.3	Structura claselor pentru Clientul Mobil	7
2.4	Structura claselor pentru API Gateway	8
2.5	Structura modulelor pentru Analysis Worker	9
2.6	Capabilități generale	10
2.7	Caracteristicile utilizatorului	10
3	Cazuri de utilizare și actori	11
3.1	Actori	11
3.2	Diagrama UML de use-case	11
4	Cerințe funcționale	12
5	Cerințe non-funcționale	13
5.1	Capacitate	13
5.2	Performanță	13
6	Interfața grafică	14
6.1	Dashboard-ul principal și selecția aplicațiilor	14
6.2	Fluxul de inițiere, monitorizare și anularea scanării	15
6.3	Gestionarea istoricului și rapoartele de securitate	16
6.4	Prezentarea rezultatului	17
6.5	Testare	18
7	Concluzii	18

Versiuni

Data	Versiune	Modificări
23.06.2026	0.1	Structura inițială a documentului.
24.06.2026	0.2	Adăugarea cerințelor specifice.
25.06.2026	0.3	Adăugarea descrierii generale, integrarea diagramei de arhitectură, definirea capabilităților, actorilor și cerințelor non-funcționale.
26.06.2026	0.4	Redactarea capabilităților, crearea diagramei UML de use-case și de secvență, detalierea cerințelor specifice.
02.07.2026	0.5	Rescrierea secțiunii contextul sistemului. Adăugarea secțiunii de modelare a datelor și a diagramei Entitate-Relație.
03.07.2026	0.6	Crearea diagramelor UML de clase. Adăugarea detaliilor de implementare și a structurii arhitecturale pentru Clientul Mobil și API Gateway.
09.07.2026	0.7	Integrarea bazei de date locale și crearea diagramei de secvențe. Finalizarea structurii arhitecturale pentru Analysis Worker și adăugarea diagramei de clase aferente.
11.07.2026	0.8	Adăugarea secțiunii de design al interfeței grafice, integrarea capturilor de ecran și documentarea fluxurilor de utilizare.

1 Introducere

1.1 Scopul documentului

Scopul acestui document este de a oferi o descriere a cerințelor funcționale și non-funcționale pentru proiectul DroidGuard, formulate clar din perspectiva utilizatorului și a modului în care acesta interacționează cu sistemul.

1.2 Scurtă descriere a proiectului

Proiectul DroidGuard este un sistem software distribuit, dedicat analizei statice de securitate a aplicațiilor Android.

Scopul principal al platformei este de a oferi utilizatorilor o soluție rapidă și eficientă pentru detectarea posibilelor amenințări.

2 Descriere generala

2.1 Contextul sistemului

Arhitectura sistemului DroidGuard este construită pe un model pe trei niveluri, conceput pentru a separa interfața utilizatorului de procesarea asincronă a fișierelor complexe. Figura 1 ilustrează fluxul de date și componentele principale ale sistemului.

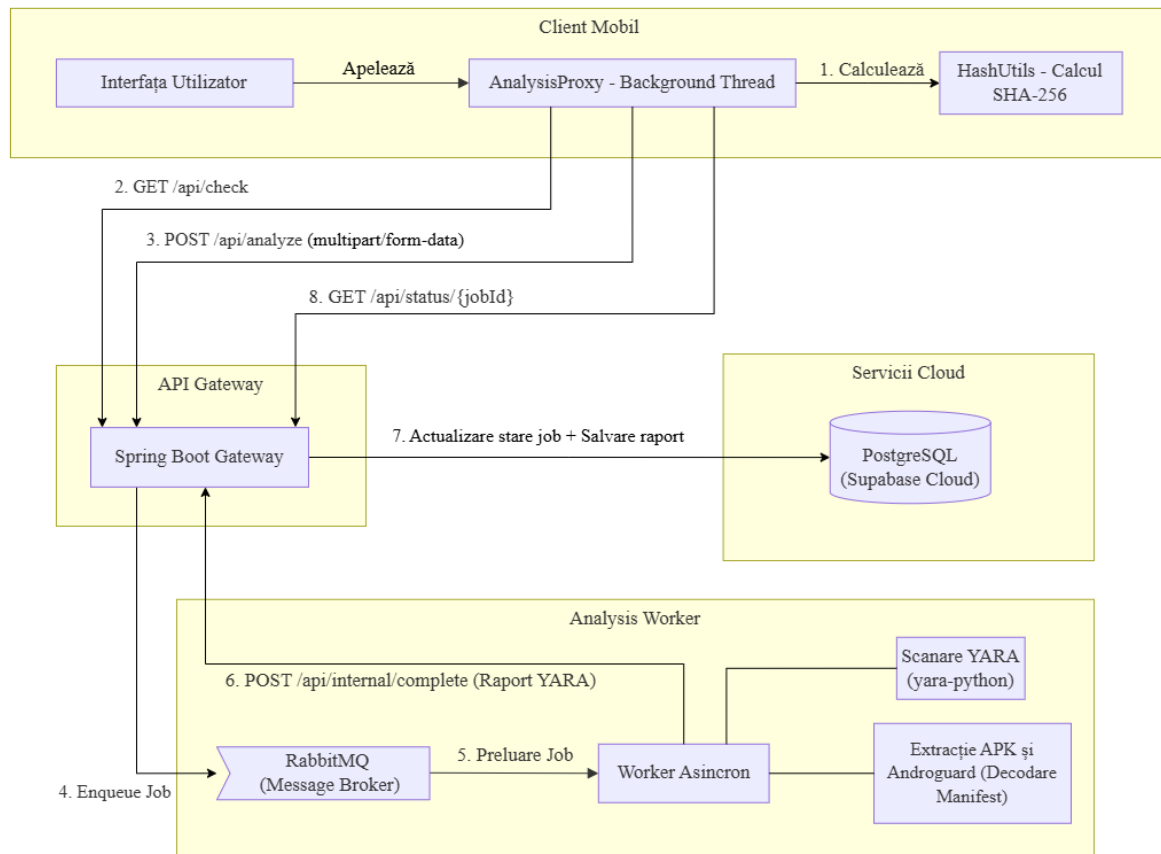


Figura 1: Arhitectura sistemului DroidGuard și fluxul de procesare client-server

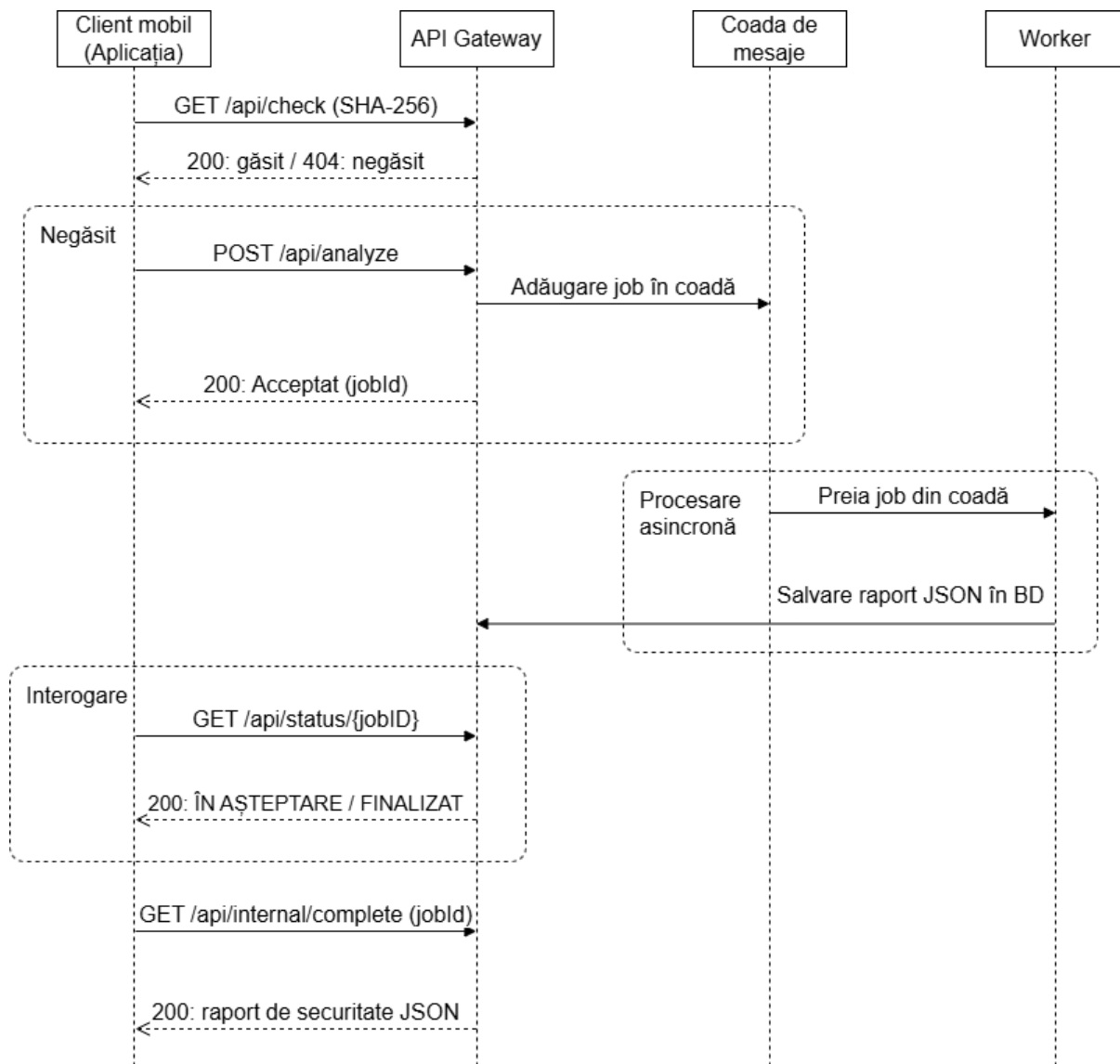


Figura 2: Diagrama de secvență a sistemului DroidGuard

În stadiul actual, sistemul DroidGuard are cele trei niveluri funcționale și integrate, după cum urmează:

- **Clientul Mobil (Android / Java):** Gestionează interfața grafică și comunicarea cu serverul. Pentru a menține fluiditatea interfeței și a preveni blocajele de tip ANR, calculul local al ampretei criptografice SHA-256 este executat asincron pe fire de execuție secundare folosind `ExecutorService`. O optimizare critică la nivel de memorie este utilizarea `URLConnection` configurat cu `setChunkedStreamingMode` pentru încărcarea aplicațiilor (prin `multipart/form-data`). Această abordare asigură transferul pachetelor APK de dimensiuni mari fără a epuiza memoria RAM a dispozitivului.
- **API Gateway (Spring Boot / Java):** Acționează ca un orchestrator central pentru rutarea sarcinilor și gestionarea stărilor. Acesta expune endpoint-uri REST

pentru verificări, interogări de status și utilizează **RabbitMQ** (model *fire-and-forget*) pentru a publica asincron sarcinile către componenta de execuție. Persistența datelor este izolată în cloud prin platforma Supabase (PostgreSQL).

- **Analysis Worker (Python):** Este componenta autonomă care execută munca intensivă. Aceasta consumă mesajele din RabbitMQ, extrage fișierele APK utilizând biblioteca **zipfile** și folosește **Androguard** pentru a decoda binarul **AndroidManifest** (fișier xml) direct în text lizibil, pregătindu-l pentru detecția de permisiuni malițioase. Scanarea propriu-zisă folosește motorul **yara-python**. Performanța worker-ului este optimizată prin:
 - **Pre-compilarea în memorie:** Regulile YARA sunt compilate global, o singură dată la pornirea serviciului, eliminând latența per-cerere.
 - **Filtrare I/O:** S-a implementat o listă de extensii (.dex, .so, .xml, .smali etc.) pentru a exclude de la citire resurse media irelevante incluse în aplicații.
 - **Procesare concurentă:** Prin scanarea pe mai multe fire.

2.2 Modelarea și persistența datelor

Având în vedere arhitectura distribuită a sistemului DroidGuard, persistența datelor este gestionată pe două niveluri interconectate: o bază de date centralizată (pentru componenta Gateway) și o memorie cache locală (pentru aplicația Android client).

Pentru a stoca la nivel global istoricul fișierelor procesate și rezultatele analizelor statice, serverul Gateway utilizează o bază de date relațională (PostgreSQL) aflată în cloud prin platforma Supabase. Figura 3 ilustrează diagrama Entitate-Relație a sistemului DroidGuard în etapa curentă.

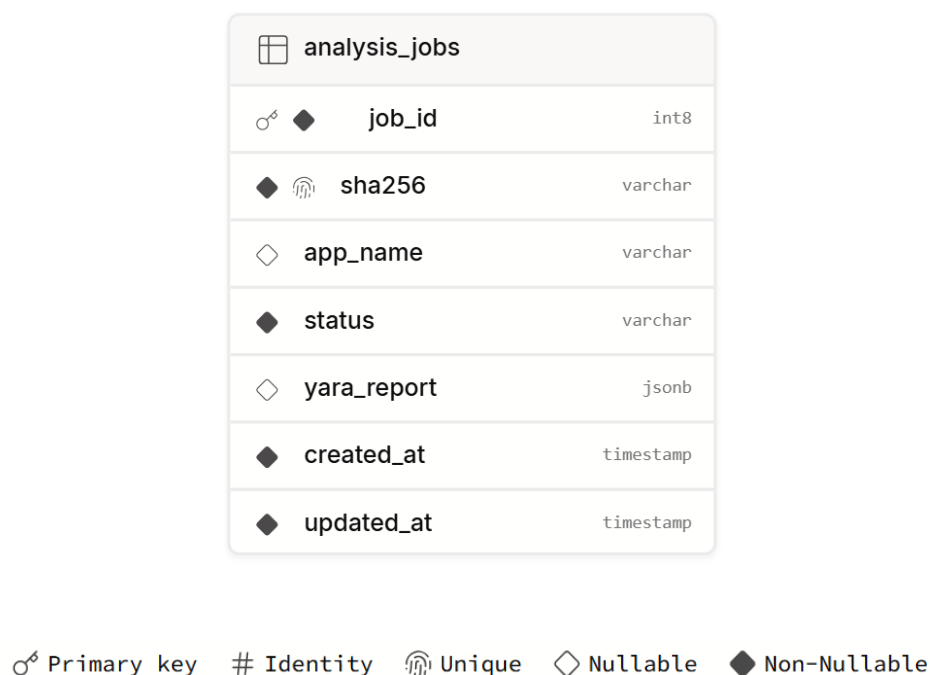


Figura 3: Diagrama E-R a bazei de date cloud și legenda simbolurilor utilizate

În loc să avem o divizare excesivă a tabelor relaționale pentru fiecare tip de vulnerabilitate detectată, rezultatul final este salvat într-o coloană de tip JSONB. Această abordare oferă flexibilitate pentru structura dinamică a rapoartelor generate de YARA și Androguard.

De asemenea, această structură reprezintă o fundație solidă pentru dezvoltarea ulterioară, cum ar fi integrarea unui sistem de gestionare a utilizatorilor. Astfel, un raport de securitate generat în urma unei scanări unice va putea fi asociat ulterior istoricului mai multor utilizatori.

Pe lângă infrastructura din cloud gestionată de serverul Spring Boot, aplicația Android utilizează o bază de date locală (SQLite, gestionată prin biblioteca Room). Figura 4 prezintă schema acestei componente.

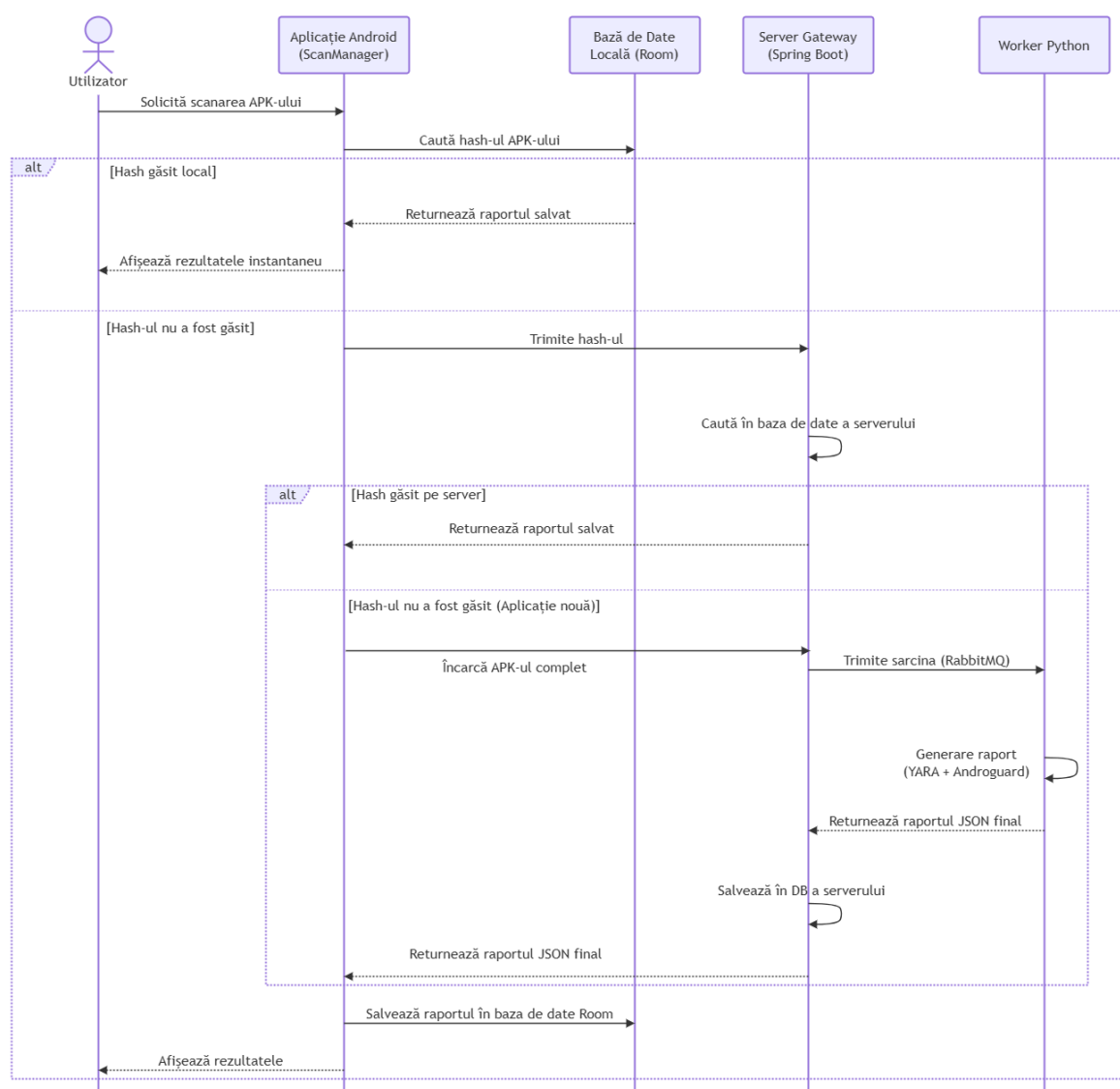


Figura 4: Diagrama de secvență a bazei de date locale utilizată de aplicația Android

Baza de date locală acționează ca un prim nivel de stocare, salvând un istoric al scanărilor efectuate direct pe telefonul utilizatorului. Aceasta permite încărcarea instantanee

a rapoartelor anterioare și elimină necesitatea unor interogări de rețea redundante către server.

2.3 Structura claselor pentru Clientul Mobil

Pentru aplicația Android, arhitectura claselor a fost proiectată cu scopul principal de a separa interfața grafică de operațiunile costisitoare din punct de vedere al prelucrării. Figura 5 ilustrează componentele principale ale clientului mobil. În cadrul diagramei au fost omise clasele, metodele și câmpurile irelevante descrierii generale.

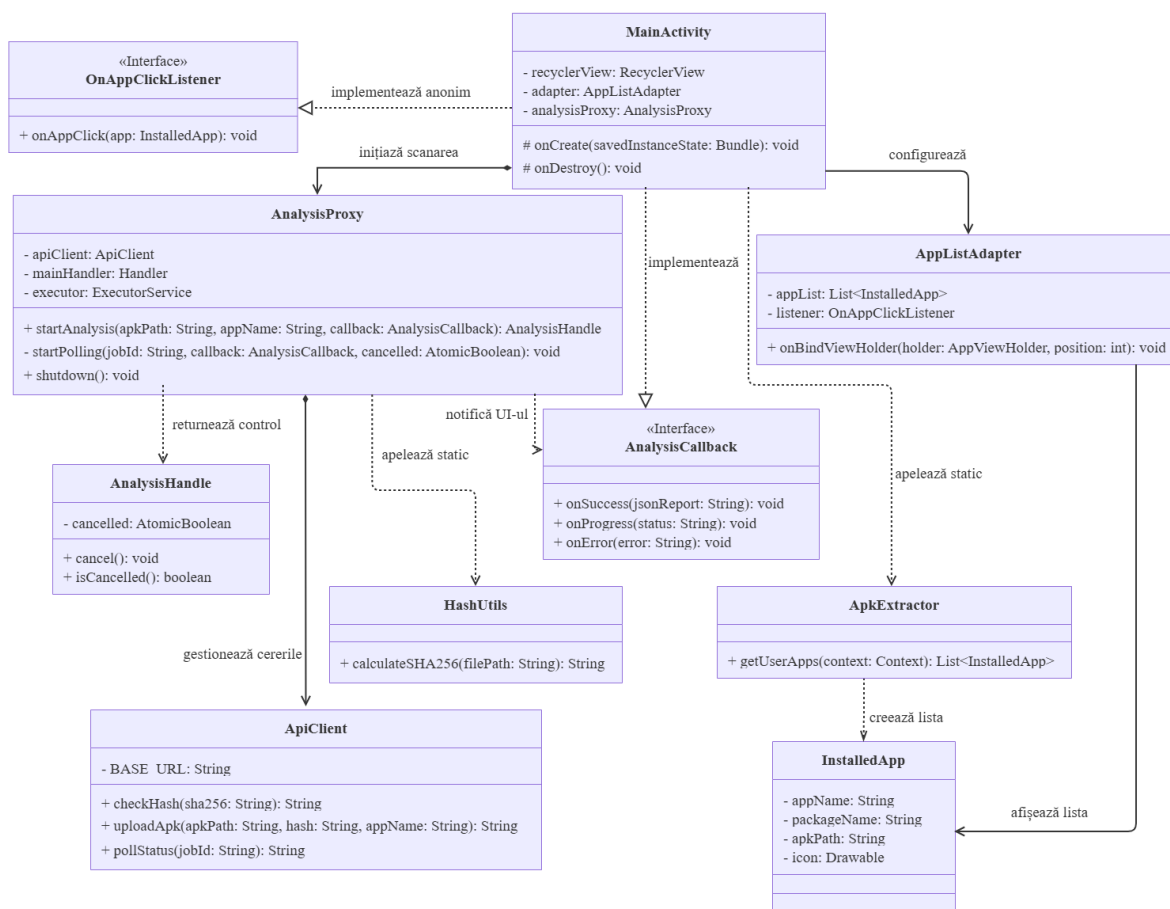


Figura 5: Diagrama principală de clase pentru Clientul Mobil

Clientul este divizat în următoarele componente logice:

- **Nivelul de interfață:** Clasa **MainActivity** este responsabilă pentru interacțiunea cu utilizatorul. Aceasta utilizează clasa utilitară **ApkExtractor** (care interoghează **PackageManager**) pentru a popula lista de obiecte de tip **InstalledApp**. Pentru afișarea dinamică, folosește **AppListAdapter** și implementează interfața **AnalysisCallback** pentru a actualiza ecranul asincron pe baza stării procesării.
- **Nivelul de coordonare asincronă:** Clasa **AnalysisProxy** acționează ca un intermediar. Pentru a preveni erorile de tip ANR, aceasta utilizează un **ExecutorService** pentru a muta calculele și cererile HTTP pe fire de execuție secundare. Aceasta

returnează un obiect **AnalysisHandle**, permițând interfeței să anuleze operațiunea la nevoie.

- **Utilitare și rețea:**

- Clasa **HashUtils** execută citirea secvențială a fișierului APK, utilizând un buffer, pentru generarea amprente SHA-256 locale.
- Clasa **ApiClient** încapsulează logica de comunicare cu serverul. Utilizarea **HttpURLConnection** configurat cu **setChunkedStreamingMode** în metoda **uploadApk** permite transmiterea fișierelor de dimensiuni mari prin **multipart/formdata** fără a le încărca integral în memoria RAM a telefonului.

2.4 Structura claselor pentru API Gateway

Componenta API Gateway a fost implementată respectând modelul specific framework-ului Spring Boot. Figura 6 ilustrează interacțiunile dintre clasele principale ale sistemului, excluzând obiectele de configurare și metodele de acces pentru a evidenția fluxul logic de date.

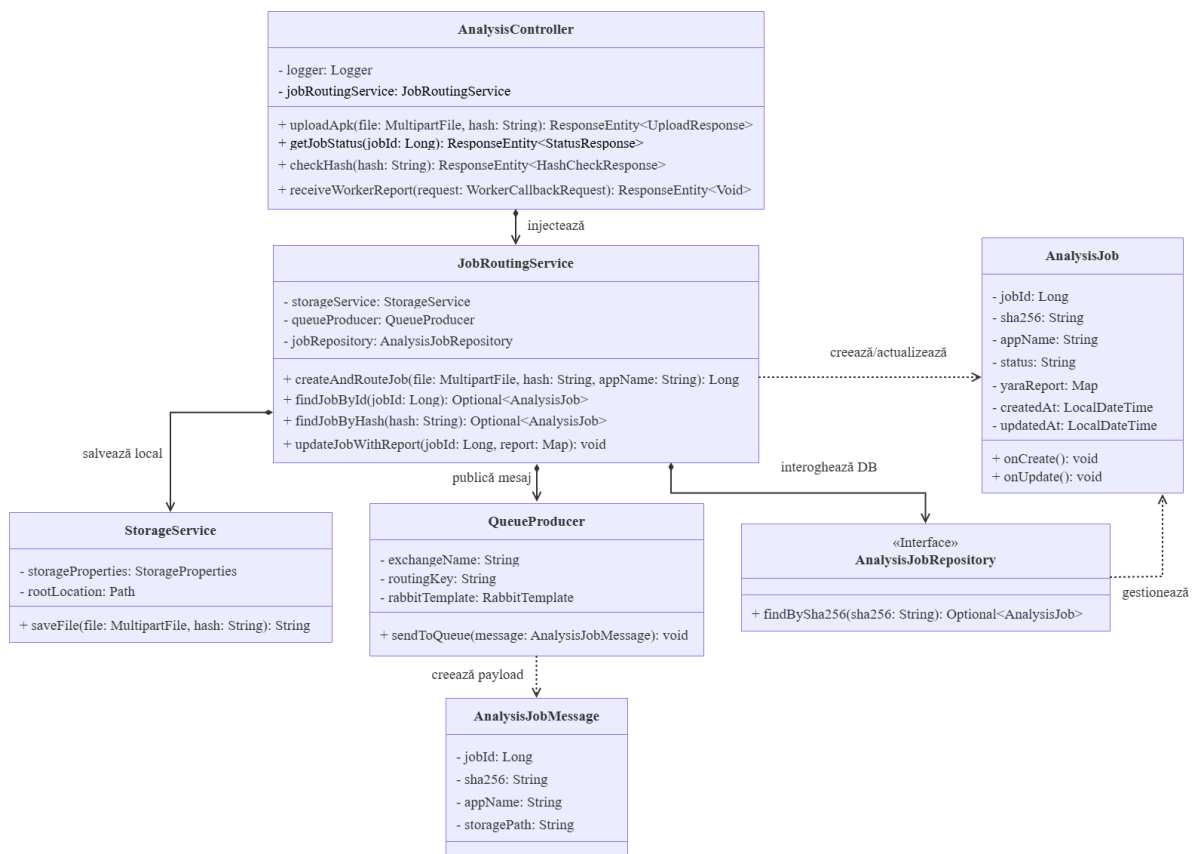


Figura 6: Diagrama principală de clase pentru API Gateway

Sistemul este structurat pe următoarele niveluri de responsabilitate:

- **Nivelul de prezentare:** Clasa **AnalysisController** gestionează exclusiv cererile HTTP externe primite de la clientul mobil și apelurile interne primite de la **Analysis**

Worker. Aceasta validează datele de intrare și delegă execuția către serviciile de business.

- **Nivelul de business:** Clasa `JobRoutingService` reprezintă nucleul logic al aplicației. Aceasta orchestrează întregul proces: interoghează baza de date, apelează `StorageService` pentru persistența locală a fișierelor APK și utilizează `QueueProducer` pentru a crea și publica obiectul `AnalysisJobMessage` în RabbitMQ.
- **Nivelul de persistență:** Interfața `AnalysisJobRepository` extinde funcționalitățile JPA pentru interogarea bazei de date (ex. `findBySha256`), gestionând entitatea `AnalysisJob` care mapează direct structura tabelului din Supabase.

2.5 Structura modulelor pentru Analysis Worker

Componenta Analysis Worker este structurată modular pentru a separa orchestrarea sarcinilor de analiza propriu-zisă a fișierelor APK. Figura 7 ilustrează fluxul logic esențial al acestei componente.

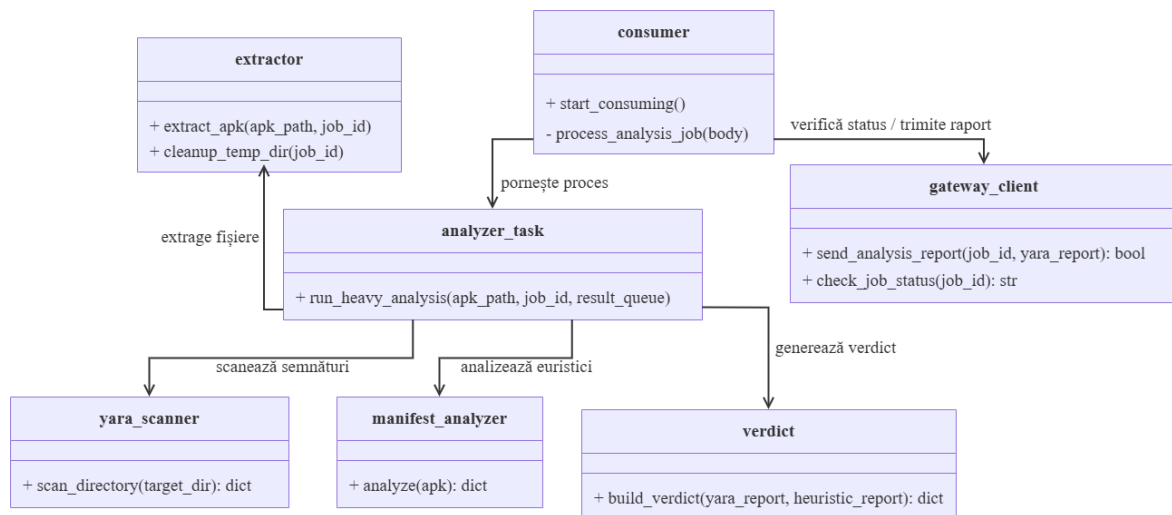


Figura 7: Diagrama principală de clase pentru Analysis Worker

Sistemul este compus din următoarele entități:

- **Consumer:** Gestionează comunicarea cu broker-ul RabbitMQ. Acesta preia mesajele, verifică dacă sarcina a fost anulată de utilizator prin `GatewayClient` și delegă analiza către `AnalyzerTask`.
- **AnalyzerTask:** Acționează ca orchestrator central. Acesta rulează analiza într-un proces izolat pentru a preveni blocarea sistemului în caz de erori critice, coordonând apelurile către modulele de procesare.
- **Nivelul de analiză (Extractor, YaraScanner, ManifestAnalyzer):** Aceasta este zona de execuție a analizelor. `Extractor` pregătește mediul de lucru și extrage fișierele APK (folosind biblioteca `zipfile`). După care transformă `AndroidManifest.xml` (format binar) în format text lizibil, folosind biblioteca `Androguard`. `YaraScanner` și `ManifestAnalyzer` extrag indicatorii de compromitere precum:

identificarea tuturor permisiunilor declarate în manifest, corelarea permisiunilor cu comportamentul potențial malițios, etc.

- **Verdict:** Colectează rezultatele intermediare și combină rezultatele din YaraScanner și ManifestAnalyzer pentru calcularea unui scor de risc final. Structurarea rezultatelor în format JSON, inclund: semnăturile detectate, permisiuni suspicioase, scor de amenințare (0.0 - 1.0). Raportul final este transmis prin GatewayClient către baza de date cloud pentru stocare.

2.6 Capabilități generale

Sistemul este conceput pentru a îndeplini următoarele funcții:

- **Extragerea aplicațiilor locale:** Identificarea aplicațiilor instalate pe dispozitivul mobil (filtrând aplicațiile de sistem) și preluarea fișierelor APK fizice corespunzătoare.
- **Eliminarea dublurilor:** Calcularea locală a ampretei SHA-256 a fișierului și interogarea bazei de date de pe server pentru a preveni încărcarea repetată a aplicațiilor deja analizate.
- **Decompilare și traducere:** Transformarea codului binar compilat (Dalvik byte-code / *.dex*) în format text (*.smali*) pe serverul de analiză, oferind astfel structura necesară pentru o scanare eficientă.
- **Scanare deterministă bazată pe reguli (YARA):** Analizarea codului sursă extras pentru a detecta semnături de cod malițios și amenințări cunoscute.
- **Raport asincron:** Generarea unor rapoarte de securitate detaliate în format JSON și livrarea acestora către clientul mobil.

2.7 Caracteristicile utilizatorului

Utilizatorul țintă al platformei DroidGuard este o persoană care deține un dispozitiv mobil cu sistem de operare Android și este interesată de verificarea de securitate a aplicațiilor instalate local. Acesta trebuie să posede doar un nivel minim de cunoștințe specifice utilizării aplicațiilor mobil standard. Sistemul este proiectat astfel încât să ascundă complet complexitatea tehnică în spatele unei interfețe grafice simple.

Cu toate acestea, pentru a beneficia la maximum de funcționalitățile sistemului, utilizatorul va trebui să aibă o înțelegere de bază a conceptelor de securitate cibernetică la citirea rapoartelor finale (de exemplu, să înțeleagă diferența dintre o aplicație sigură și una nesigură).

3 Cazuri de utilizare și actori

3.1 Actori

În cadrul aplicației, au fost identificați următorii actori:

- **Utilizator Android:** Reprezintă persoana fizică ce utilizează aplicația mobilă. Acest actor inițiază procesul de verificare, aprobă încărcarea fișierelor și vizualizează rapoartele de securitate. Interacțiunea sa are loc prin intermediul interfeței grafice.
- **API Gateway:** Reprezintă componenta centrală care primește cererile de la clientul mobil. Acționează ca un intermediar care gestionează interogările bazei de date, acceptă fișierele și delegă sarcinile mai departe.
- **Analysis Worker:** Reprezintă componenta asincronă care execută analiza intensivă. Acționează autonom odată ce preia un mesaj din coada de mesaje, coordonând decompilarea fișierelor și rularea motorului YARA.

3.2 Diagrama UML de use-case

Diagrama de mai jos ilustrează interacțiunile dintre actorii identificați și cazurile de utilizare specifice sistemului DroidGuard.

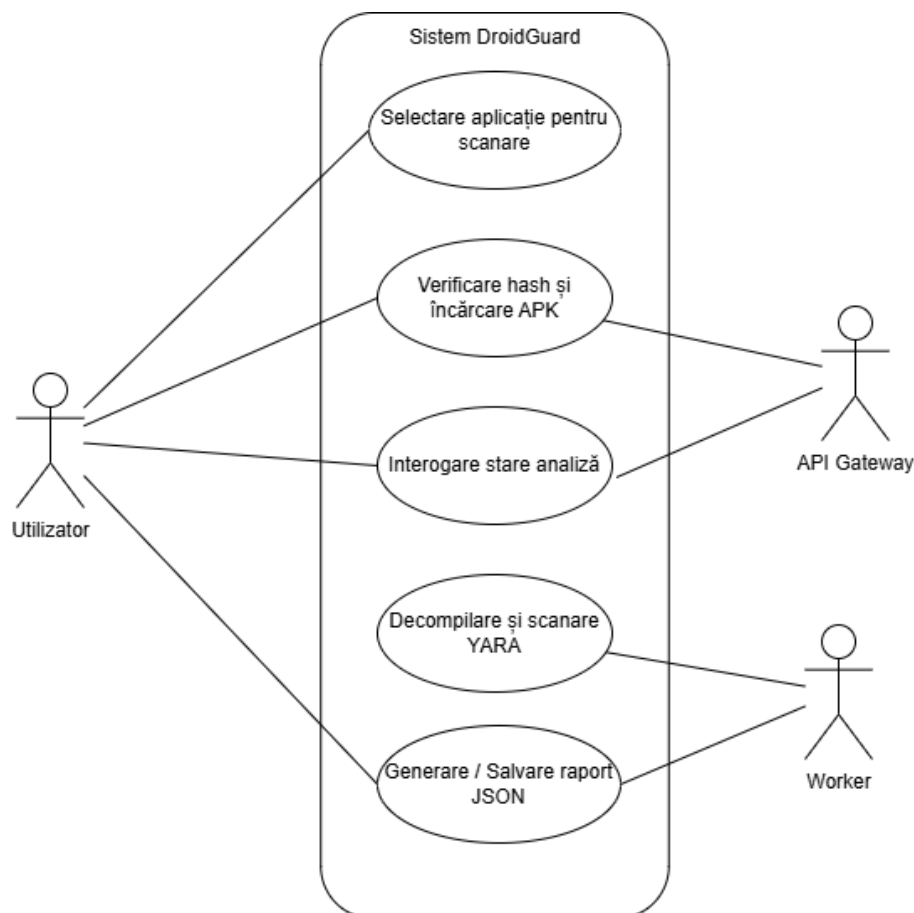


Figura 8: Diagrama de cazuri de utilizare a sistemului DroidGuard

4 Cerințe funcționale

Nr.	Rol	Descrierea cerinței	Motivație	Prioritate
1	Utilizator	Sistemul va afișa o listă a aplicațiilor instalate de utilizator, excluzând pachetele de sistem.	Facilitarea selecției rapide a aplicației țintă.	Maximă
2	Sistem	Calculare și interogare automată a amprentei criptografice SHA-256 a aplicației pe server, anterior oricărui transfer de date.	Optimizarea lățimii de bandă și furnizarea unui răspuns instantaneu în cazul fișierelor existente.	Maximă
3	Utilizator	Aplicația client va permite transferul securizat al fișierului APK fizic către infrastructura Droid-Guard.	Externalizarea procesului de analiză statică.	Maximă
4	Utilizator	Utilizatorul va avea posibilitatea de a monitoriza stadiul de procesare al fișierului transmis.	Asigurarea transparenței privind progresul sarcinii asincrone de decompilare și scanare executate pe server.	Maximă
5	Utilizator	Interfața grafică va prezenta utilizatorului raportul final de securitate într-un format structurat și lizibil.	Furnizarea datelor necesare pentru identificarea amenințărilor detectate, permițând o evaluare informată a riscurilor.	Maximă

5 Cerințe non-funcționale

Definirea constrângerilor și atributelor de calitate ale sistemului sunt esențiale pentru asigurarea unei experiențe optime pentru utilizator și a unei operări sigure pe server.

5.1 Capacitate

- **Gestionarea concurenței:** Datorită arhitecturii decuplate bazate pe cozi de mesaje, sistemul trebuie să poată prelua și stoca în așteptare un număr teoretic nelimitat de cereri simultane de analiză, evitând pierderea datelor sau suprasolicitarea Gateway-ului în perioadele de vârf.
- **Dimensiunea maximă a fișierelor:** API Gateway-ul trebuie să permită transferul unor fișiere APK cu o dimensiune de până la 100 MB, acoperind astfel majoritatea aplicațiilor standard de Android.
- **Scalabilitate orizontală:** Arhitectura componentei de backend trebuie să permită adăugarea facilă de noi instanțe pentru a mări capacitatea globală de analiză a sistemului, fără a necesita modificări la nivelul codului sursă.

5.2 Performanță

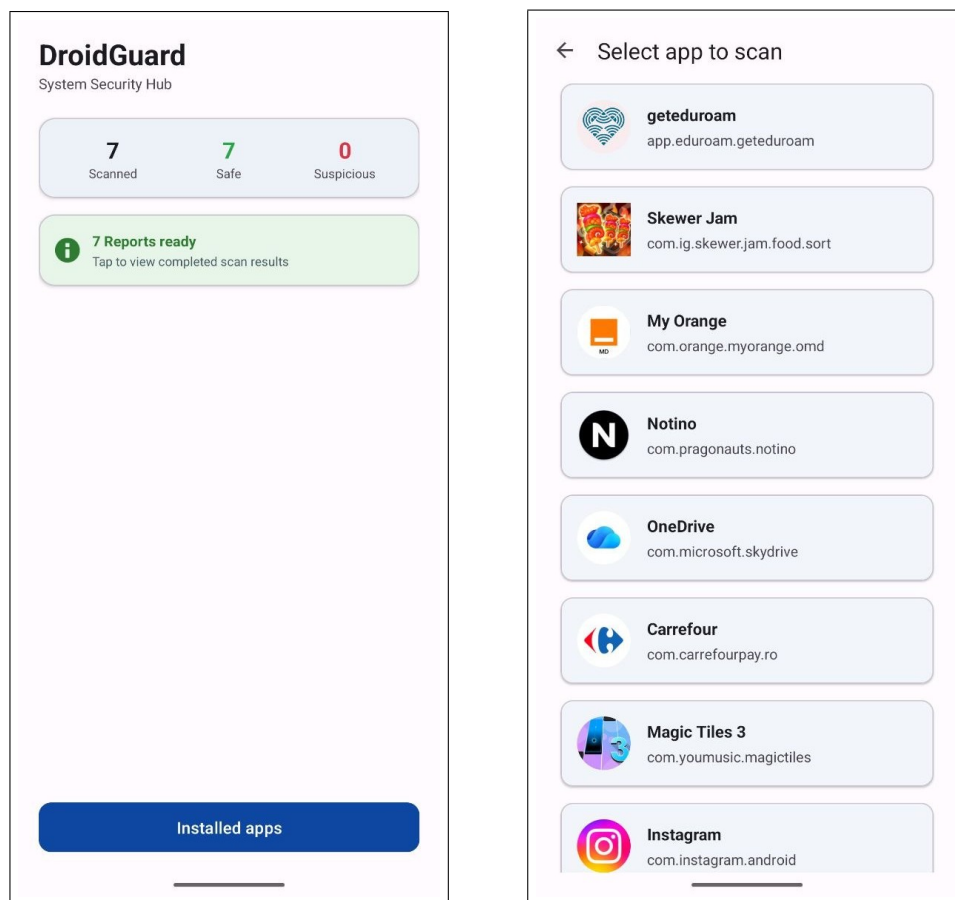
- **Timp de răspuns sincron:** Răspunsul pentru cererile HTTP ușoare, precum verificarea amprente SHA-256 și interogarea statusului analizei, trebuie să fie returnat în mai puțin de 500 de milisecunde, asigurând flexibilitate interfeței mobile.
- **Timp de execuție asincronă:** Procesul complet de analiză pe server (decompilarea și scanarea fișierelor) ar trebui să se finalizeze, în medie, în 1-3 minute, durata variind strict în funcție de dimensiunea și complexitatea bytecode-ului analizat.
- **Eficiența resurselor pe dispozitivul mobil:** Calculele criptografice și transferul fișierelor trebuie executate exclusiv pe un fir de execuție secundar. Acest lucru garantează că interfața utilizatorului nu se blochează, prevenind erorile de tip ANR (*Application Not Responding*) și menținând consumul de baterie la un nivel minim.

6 Interfața grafică

Interfața grafică a aplicației DroidGuard pune accent pe simplitate, claritate și accesibilitate. Complexitatea tehnică a analizei malware este abstractizată în spatele unui design intuitiv.

6.1 Dashboard-ul principal și selecția aplicațiilor

Ecranul principal oferă utilizatorului o imagine de ansamblu asupra stării de securitate a dispozitivului, urmat de meniul de selecție a aplicațiilor țintă.



Ecran principal

Lista aplicațiilor instalate

Figura 9: Punctul de intrare în aplicație și selecția aplicației pentru analiză

- **Ecranul principal:** Afășează un panou de statistici globale. Mai jos se află informația despre rapoartele finalizate. Navigarea către începerea unei noi scanări se face prin butonul „Installed apps”, plasat în partea de jos a ecranului.
- **Ecranul de selecție aplicații:** Listează toate aplicațiile instalate pe dispozitiv (cu excepția celor de sistem). Fiecare aplicație este reprezentat printr-un card ce include iconița originală a aplicației, numele public și denumirea tehnică a pachetului.

6.2 Fluxul de inițiere, monitorizare și anularea scanării

Pentru a preveni acțiunile accidentale și a oferi control total utilizatorului asupra consumului de date și resurse, sistemul implementează dialoguri explicite de confirmare și opțiuni de întrerupere a execuției.

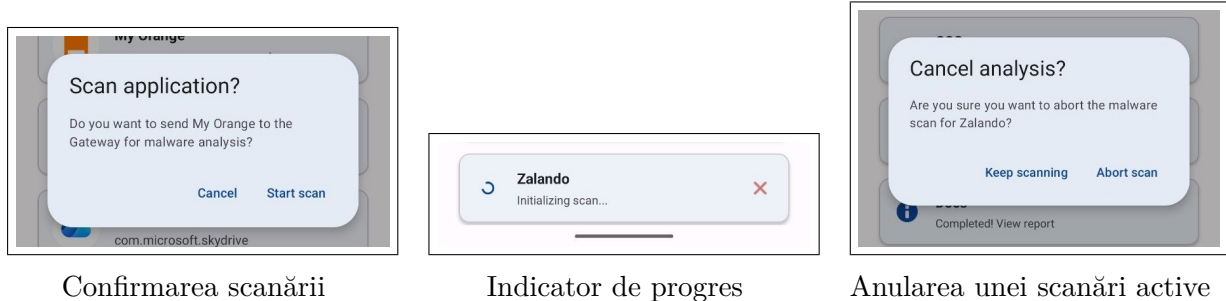
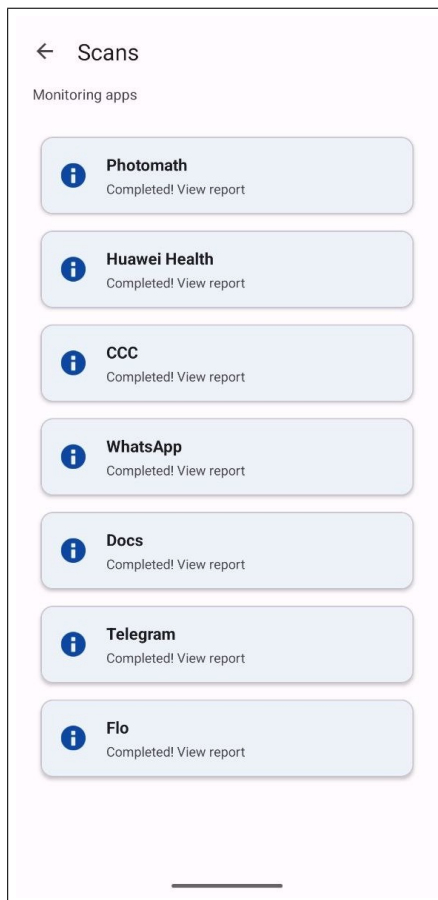


Figura 10: Mecanismele de control al fluxului de analiză

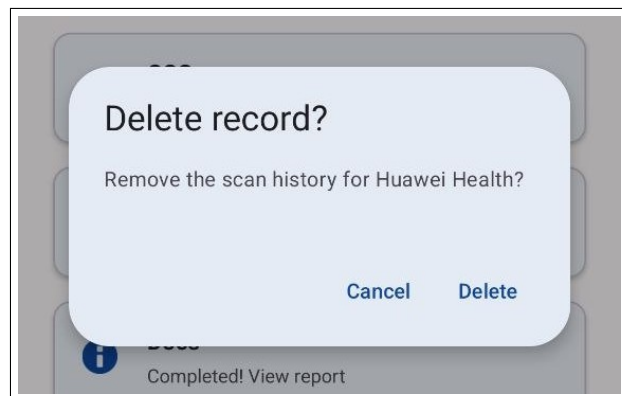
- **Confirmarea scanării:** La selectarea unei aplicații, un dialog avertizează utilizatorul că datele vor fi trimise către Gateway pentru analiză.
- **Feedback vizual:** Pe durata comunicării cu serverul, interfața afișează un indicator de progres. Această componentă include un buton roșu de anulare.
- **Gestionarea anulării:** La selectarea unei aplicații, un dialog avertizează că scanarea va fi oprită.

6.3 Gestionarea istoricului și rapoartele de securitate

Rezultatele finale sunt salvate local, eliminând dependența de conexiunea la internet pentru revizualizarea analizelor anterioare.



Lista rapoartelor salvate



Ștergerea unei înregistrări

Figura 11: Interfața pentru gestionarea istoricului de scanări locale

Aici se află lista a tuturor analizelor finalizate. Totodată, interfața oferă opțiunea de a elimina istoricul oricărei aplicații analizate anterior, aplicația scanată fiind în continuare prezentă în baza de date globală.

6.4 Prezentarea rezultatului

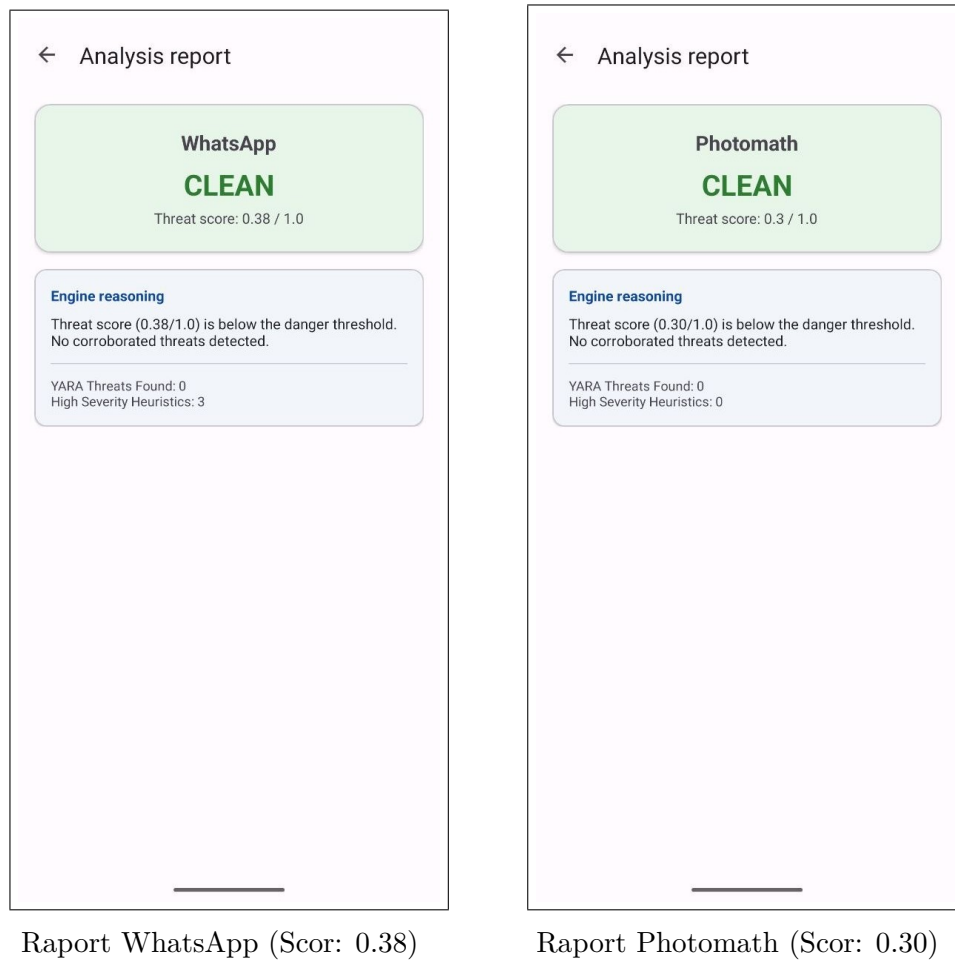


Figura 12: Exemple de rapoarte de analiză detaliate

Fiecare raport este structurat în două secțiuni:

1. **Zona de verdict:** Folosește un cod de culoare pentru fiecare rezultat. Sub verdict este afișat scorul de amenințare normalizat între 0.0 și 1.0.
2. **Raționamentul motorului:** Oferă o explicație textuală a deciziei și un rezumat al indicatorilor descoperiți de backend, cum ar fi amenințările YARA găsite și numărul de euristici.

6.5 Testare

Sistemul a fost testat pe un set variat de aplicații comerciale standard. Timpul mediu de analiză variază în funcție de aplicație (de obicei, câteva minute). Tabelul 1 sumarizează rezultatele reale obținute în urma testării pe dispozitiv, vizibile și în capturile de ecran.

Aplicație	Verdict	Scorul	Euristici înalte
WhatsApp	CLEAN	0.38 / 1.0	3
Photomath	CLEAN	0.30 / 1.0	0

Tabela 1: Rezultate obținute în urma testării a sistemului

Deși aplicații precum WhatsApp prezintă comportamente euristice marcate cu severitate înaltă (datorită accesului extins la sistem necesar funcționării legitime), motorul a clasificat corect aplicația ca fiind benign, menținând scorul sub pragul critic de 0.4.

Notă privind acoperirea testelor: În această etapă a proiectului, testarea s-a concentrat exclusiv pe aplicații benigne pentru a calibra motorul de decizie. Evaluarea performanței și acurateții sistemului împotriva unor aplicații malițioase este planificată pentru săptămâna următoare.

7 Concluzii

Proiectul DroidGuard reprezintă o soluție modernă și scalabilă pentru analiza statică de securitate a aplicațiilor Android. Prin adoptarea unei arhitecturi distribuite (Client Mobil, API Gateway și Analysis Worker), sistemul reușește să decupleze interfața utilizatorului de procesarea intensivă necesară decompilării și scanării fișierelor.

Această abordare nu doar că previne suprasolicitarea resurselor hardware ale dispozitivului mobil, dar asigură și un timp de răspuns optim.

În concluzie, DroidGuard oferă utilizatorilor un instrument accesibil pentru identificarea potențialelor amenințări, ascunzând complexitatea tehnică în spatele unei interfețe grafice intuitive și pune o bază solidă pentru viitoarele dezvoltări în domeniul securității cibernetice mobile.